

Macro-Architectural Routing, Parameter-Efficient Adaptation, and Accelerated Inference Dynamics in Modern Transformers

Sahki Abderrahemane

AI Engineer & Deep Learning Enthusiast

June 2026

Contents

1	Introduction	2
2	Transformer Architectural Routing Paradigms	3
2.1	Graph Routing Intuition	3
2.2	Comparative Architectural Analysis	4
2.2.1	BERT: Encoder-Only Bidirectional Mask	4
2.2.2	GPT: Decoder-Only Causal Mask	4
2.2.3	T5: Encoder-Decoder Cross-Attention	4
2.3	Visualizing Attention Masks	5
2.4	Architecture Overview Diagram	5
2.5	Code Implementation: Causal Mask	5
3	Parameter-Efficient Fine-Tuning & Quantization	7
3.1	Intrinsic Rank Hypothesis and LoRA	7
3.2	Initialization Mechanics: Why A Random, B Zero	7
3.3	QLoRA: 4-bit NormalFloat Quantization	8
3.4	Code Implementation: LoRA Linear Layer	9
4	Tokenization Internals & Inference Engineering	10
4.1	Subword Tokenization Algorithms	10
4.1.1	Byte-Pair Encoding (BPE)	10
4.1.2	WordPiece	10
4.1.3	SentencePiece	10
4.2	KV-Cache Dynamics in Autoregressive Generation	11
4.3	Code Implementation: KV-Cache Usage in Attention	12
5	Conclusion	12

1 Introduction

Modern large language models (LLMs) derive their power from a combination of architectural routing choices, efficient adaptation strategies, tokenization granularity, and inference-time optimizations. Each of these components has a deep mathematical foundation, and their interaction governs the performance, memory footprint, and deployability of models with billions of parameters. This report provides a rigorous end-to-end analysis of these interconnected systems.

We begin by formalizing Transformer attention as a graph routing problem. The attention mask acts as an adjacency matrix that defines valid edges in a fully connected graph of token nodes. We compare the three dominant routing paradigms: the open mask of BERT for bidirectional understanding, the causal mask of GPT for autoregressive generation, and the encoder-decoder cross-attention of T5 for conditional generation. A complete mathematical proof demonstrates why additive $-\infty$ masking forces softmax attention weights to exactly zero on illegal edges, preventing information leakage. PyTorch implementations are provided throughout.

The second section tackles parameter-efficient fine-tuning. We derive the Low-Rank Adaptation (LoRA) decomposition $\mathbf{W}_0 + \mathbf{B}\mathbf{A}$ from the intrinsic rank hypothesis, and explain the critical initialization strategy ($\mathbf{A}$ random with Kaiming uniform, $\mathbf{B}$ exactly zero) with a gradient-flow analysis showing what happens when this configuration is inverted. We then extend the discussion to QLoRA, which quantizes frozen weights to 4-bit NormalFloat (NF4) and applies double quantization to scaling constants, enabling fine-tuning of 65B-parameter models on a single GPU.

The final section addresses tokenization internals and inference engineering. We contrast Byte-Pair Encoding (BPE), WordPiece, and SentencePiece, with particular attention to byte-level fallback and its disproportionate impact on non-Latin scripts. We then derive the KV-cache mechanism mathematically, showing how caching past key and value projections transforms autoregressive generation from an $O(T^2)$ compute bottleneck to an $O(1)$ per-step operation at the cost of linear memory growth. Together, these topics constitute the engineering backbone of the modern LLM lifecycle.

2 Transformer Architectural Routing Paradigms

2.1 Graph Routing Intuition

A Transformer can be interpreted as a fully connected directed graph with T nodes, each representing a token embedding. The attention mechanism computes a weighted sum of values, where the weight a_{ij} from token i to token j is obtained by applying softmax over the query-key dot-product logits. By adding an additive mask $\mathbf{M} \in \mathbb{R}^{T \times T}$ to the logits, we can manually delete or keep edges. The masked attention operation is

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{M}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right) \mathbf{V}. \quad (1)$$

Setting $M_{ij} = -\infty$ forces the corresponding attention weight to zero after softmax, because $\lim_{x \rightarrow -\infty} e^x = 0$. Thus the mask acts as an adjacency matrix that hard-wires the permitted information flow. Figure 1 illustrates this concept.

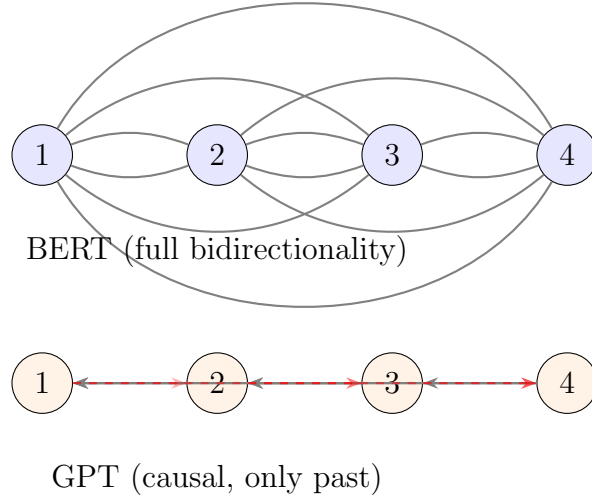


Figure 1: Attention routing. Left: BERT uses a fully open (zeros) mask, allowing information flow between any pair of tokens. Right: GPT uses a causal mask, blocking edges from future tokens (red dashed).

2.2 Comparative Architectural Analysis

2.2.1 BERT: Encoder-Only Bidirectional Mask

BERT employs a zero mask, $\mathbf{M} = \mathbf{0}$. Every token can attend to every other token in both directions, allowing the model to build rich bidirectional representations. This makes it ideal for classification and understanding tasks. The absence of autoregressive constraints also permits fully parallel computation over all positions, making training highly efficient.

2.2.2 GPT: Decoder-Only Causal Mask

Autoregressive language models like GPT must prevent each token from attending to future positions. This is achieved with a lower-triangular mask that sets $M_{ij} = -\infty$ for $j > i$. The mask is defined as

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases} \quad (2)$$

Mathematical proof of zero leakage: For a given query $\mathbf{q}_i$, the attention logit for a future key $\mathbf{k}_j$ ($j > i$) becomes $\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} + (-\infty) = -\infty$. The softmax weight is then

$$a_{ij} = \frac{e^{-\infty}}{\sum_k e^{\text{logit}_k}} = \frac{0}{\text{positive sum}} = 0. \quad (3)$$

Thus no information from future tokens can affect the output, ensuring strict causality. This is essential for autoregressive generation, where each new token depends only on previously generated tokens.

2.2.3 T5: Encoder-Decoder Cross-Attention

The T5 model pairs a bidirectional encoder with a causal decoder. The encoder uses an all-zero mask, processing the source sequence in parallel. The decoder uses a causal self-attention mask (like GPT), but additionally includes a cross-attention sub-layer where the queries come from the decoder hidden states, and the keys and values come from the encoder output. The cross-attention mask is fully open ($\mathbf{M} = \mathbf{0}$), allowing every decoder position to attend to the entire encoded source. This design is powerful for conditional generation tasks such as translation and summarization.

2.3 Visualizing Attention Masks

Figure 2 contrasts the open mask (BERT) and causal mask (GPT) as grid matrices. For a sequence length $T = 5$, the open mask is all zeros, while the causal mask has $-\infty$ (shown in red) in the upper triangle.

BERT Mask (Open)					GPT Mask (Causal)				
0	0	0	0	0	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$
0	0	0	0	0	0	0	$-\infty$	$-\infty$	$-\infty$
0	0	0	0	0	0	0	0	$-\infty$	$-\infty$
0	0	0	0	0	0	0	0	0	$-\infty$
0	0	0	0	0	0	0	0	0	0

Figure 2: Attention mask matrices for $T = 5$. Left: BERT’s open mask (all zeros). Right: GPT’s causal mask (lower-triangular zeros, upper-triangular $-\infty$). Red cells are prohibited connections.

2.4 Architecture Overview Diagram

Figure 3 summarizes the three main Transformer paradigms: encoder-only (BERT), decoder-only (GPT), and encoder-decoder (T5). The flow of information and the type of attention used are indicated.

2.5 Code Implementation: Causal Mask

Listing 1 creates a lower-triangular causal mask using `torch.tril` and `masked_fill`. We first build a boolean mask that is `True` for allowed positions and `False` for forbidden ones; then we fill the forbidden entries with $-\infty$.

```
1 import torch
2
3 def create_causal_mask(seq_len: int, device: torch.device =
4     None) -> torch.Tensor:
5     """
6     Returns a (seq_len, seq_len) additive mask where
7     allowed positions are 0.0 and masked positions are -inf.
8     """
9     # Lower-triangular boolean: allowed = True, forbidden =
10     False
```

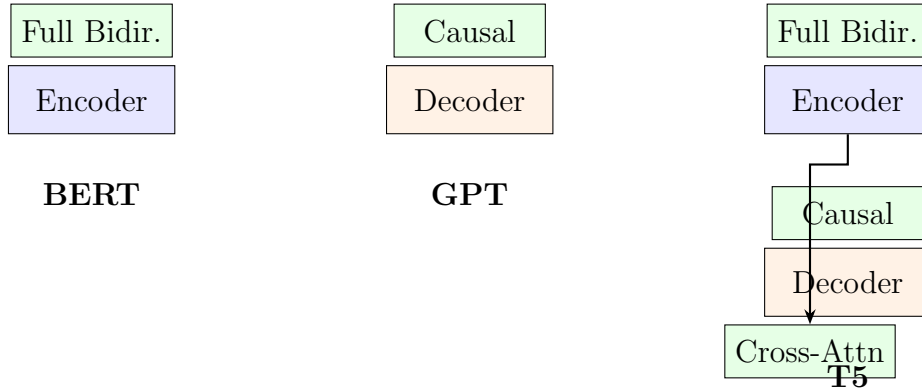


Figure 3: High-level architectures. BERT uses only an encoder with full bidirectional attention. GPT uses a decoder with causal self-attention. T5 pairs an encoder (bidirectional) with a decoder that has causal self-attention and cross-attention to the encoder output.

```

9      tril_bool = torch.tril(torch.ones(seq_len, seq_len, dtype
10                                     =torch.bool, device=device))
11      # We want to fill False (upper triangle) with -inf
12      # masked_fill replaces where mask==True; so we invert the
13      # boolean
14      mask = torch.zeros(seq_len, seq_len, device=device).
15      masked_fill(~tril_bool, float('-inf'))
16      return mask
17
18 # Example
19 T = 4
20 causal_mask = create_causal_mask(T)
21 print(causal_mask)
22 # tensor([[ 0., -inf, -inf, -inf],
23 #         [ 0.,  0., -inf, -inf],
24 #         [ 0.,  0.,  0., -inf],
25 #         [ 0.,  0.,  0.,  0.]])

```

Listing 1: Creation of a causal attention mask using `torch.tril` and `masked_fill`.

3 Parameter-Efficient Fine-Tuning & Quantization

3.1 Intrinsic Rank Hypothesis and LoRA

The large pre-trained weight matrices of Transformers are highly over-parameterized for any single downstream task. The **intrinsic rank hypothesis** states that the task-specific weight updates $\Delta \mathbf{W}$ lie in a low-dimensional subspace. Low-Rank Adaptation (LoRA) exploits this by parameterizing the update as a product of two low-rank matrices:

$$\mathbf{W}_{\text{new}} = \mathbf{W}_0 + \Delta \mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r} \mathbf{B} \mathbf{A}, \quad (4)$$

where $\mathbf{W}_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is frozen, $\mathbf{A} \in \mathbb{R}^{r \times d_{\text{in}}}$, $\mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r}$, and $r \ll \min(d_{\text{in}}, d_{\text{out}})$. The scalar α is a scaling factor (often 16), and only $\mathbf{A}$ and $\mathbf{B}$ receive gradients, dramatically reducing the number of trainable parameters and optimizer memory.

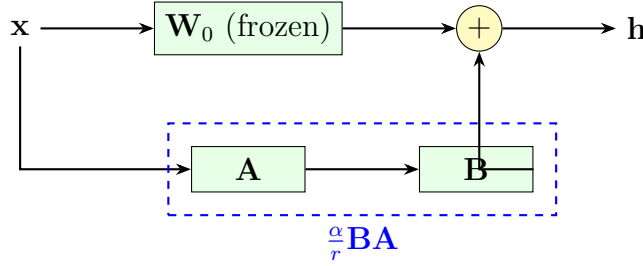


Figure 4: LoRA decomposition: the output is the sum of the frozen pre-trained weight path and the low-rank adapter path. Only the matrices $\mathbf{A}$ and $\mathbf{B}$ are updated during fine-tuning.

3.2 Initialization Mechanics: Why A Random, B Zero

To ensure that the fine-tuning starts from the exact pre-trained model, the adapter contribution must be zero at initialization. This is achieved by setting one of the two matrices to zero. The standard LoRA initialization is:

- $\mathbf{A}$ is initialized with `nn.init.kaiming_uniform_` to preserve forward-activation variance,
- $\mathbf{B}$ is initialized to $\mathbf{0}$.

Consequently, $\mathbf{B}\mathbf{A}\mathbf{x} = \mathbf{0}$ $\mathbf{A}\mathbf{x} = \mathbf{0}$, and the first forward pass equals $\mathbf{W}_0\mathbf{x}$.

What happens if we invert this configuration? Suppose we set $\mathbf{A} = \mathbf{0}$ and $\mathbf{B}$ random. The initial adapter output is also zero, so the pre-trained behaviour is preserved. However, the backward pass reveals a critical difference. The gradient w.r.t. $\mathbf{A}$ is

$$\frac{\partial L}{\partial \mathbf{A}} = \frac{\alpha}{r} \mathbf{B}^\top \frac{\partial L}{\partial \mathbf{h}} \mathbf{x}^\top. \quad (5)$$

Since $\mathbf{B}$ is random and non-zero, $\mathbf{A}$ receives non-zero gradients and begins to learn immediately. Meanwhile, $\mathbf{B}$ also receives gradients, but they are initially zero because $\mathbf{A} = \mathbf{0}$ kills the gradient flowing to $\mathbf{B}$. Specifically:

$$\frac{\partial L}{\partial \mathbf{B}} = \frac{\alpha}{r} \frac{\partial L}{\partial \mathbf{h}} (\mathbf{A}\mathbf{x})^\top = \mathbf{0}. \quad (6)$$

Thus all adaptation initially focuses on $\mathbf{A}$. This asymmetric learning may lead to slower convergence or poorer solutions because the random projection $\mathbf{B}$ is not guided in the first steps. In contrast, the standard scheme ($\mathbf{A}$ random, $\mathbf{B}$ zero) allows $\mathbf{B}$ to receive immediate non-zero gradients while $\mathbf{A}$ is frozen at random, effectively learning which directions in the random projection are useful. Empirically, the standard initialization consistently yields better results and is universally adopted.

3.3 QLoRA: 4-bit NormalFloat Quantization

QLoRA pushes LoRA’s memory savings further by quantizing the frozen pre-trained weights to 4 bits. The core innovations are:

NormalFloat4 (NF4). The pre-trained weights are assumed to follow an approximately normal distribution. The NF4 data type partitions the standard normal distribution into $2^4 = 16$ equally probable intervals, obtaining the quantile values $q_0, q_1, \dots, q_{15}$ of the cumulative distribution function Φ . A weight w is quantized to the index of the nearest quantile. This minimizes information loss compared to uniform quantization when the data is Gaussian.

Double Quantization (DQ). The per-block scaling constants (one per small block of weights) are themselves quantized to 8-bit floats, further reducing the memory footprint.

On-the-fly dequantization. During the forward and backward passes, the stored 4-bit indices are dequantized back to FP16 as needed:

$$w_{\text{FP16}} = s \cdot (\text{NF4_table}[\text{id}x]), \quad (7)$$

where s is the block scaling factor. This dequantization happens in the GPU's fast memory, but the arithmetic is still performed in FP16. The main bottleneck is the additional memory bandwidth and compute spent on reading and decompressing the 4-bit representations. Despite this overhead, QLoRA enables fine-tuning a 65B model on a single 48 GB GPU.

3.4 Code Implementation: LoRA Linear Layer

Listing 2 gives a production-ready PyTorch implementation of a LoRA-adapted linear layer with the correct initialization scheme.

```

1 import torch
2 import torch.nn as nn
3 import math
4
5 class LoRALinear(nn.Module):
6     def __init__(self, in_features: int, out_features: int, r
7         : int = 8,
8             alpha: float = 16.0, dropout: float = 0.0):
9         super().__init__()
10        self.r = r
11        self.alpha = alpha
12        self.scaling = alpha / r
13
14        # Frozen pre-trained weight
15        self.W0 = nn.Linear(in_features, out_features, bias=
16            False)
17        for param in self.W0.parameters():
18            param.requires_grad = False
19
20        # Low-rank adapters
21        self.A = nn.Parameter(torch.empty(r, in_features))
22        self.B = nn.Parameter(torch.empty(out_features, r))
23        self.dropout = nn.Dropout(dropout) if dropout > 0
24            else nn.Identity()
25
26        self._init_parameters()
27
28    def _init_parameters(self):
29        # A: Kaiming uniform for stable forward activation
30        # variance
31        nn.init.kaiming_uniform_(self.A, a=math.sqrt(5))
32        # B: Zero so that BA = 0 at initialization
33        nn.init.zeros_(self.B)
34
35    def forward(self, x: torch.Tensor) -> torch.Tensor:
36        base = self.W0(x)
37        adapted = self.dropout(x @ self.A.T) @ self.B.T

```

```
34 |         return base + self.scaling * adapted
```

Listing 2: Custom LoRA linear layer with correct initialization.

4 Tokenization Internals & Inference Engineering

4.1 Subword Tokenization Algorithms

Tokenization converts raw text into a sequence of integers. Modern LLMs rely on subword algorithms that balance vocabulary size and out-of-vocabulary (OOV) handling. The choice of tokenizer has profound implications for sequence length, model efficiency, and multilingual performance.

4.1.1 Byte-Pair Encoding (BPE)

BPE starts with a character-level vocabulary and iteratively merges the most frequent pair of adjacent symbols. Merging is guided solely by co-occurrence frequency. It is used by GPT models. The vocabulary is deterministic given a training corpus and the number of merge operations. BPE has the advantage of simplicity and handles rare words by splitting them into frequent subword units.

4.1.2 WordPiece

Originally proposed for BERT, WordPiece also builds a subword vocabulary by merging pairs, but the criterion is based on likelihood maximization: at each step, the pair (a, b) that maximizes the increase in the training data likelihood under the current language model is merged. This tends to produce a vocabulary that is more linguistically coherent, but requires a more expensive training procedure. The likelihood objective is:

$$\text{score}(a, b) = \frac{\text{count}(a, b)}{\text{count}(a) \cdot \text{count}(b)}, \quad (8)$$

where higher scores indicate pairs that occur together more often than expected by chance.

4.1.3 SentencePiece

SentencePiece treats the input as a raw byte stream, completely bypassing language-specific pre-tokenization (e.g., splitting on spaces). It implements

both BPE and unigram language model tokenization. By operating directly on Unicode bytes, it ensures a lossless tokenization that can handle any language, even those without explicit word boundaries. A key component is **byte-level fallback**: if a token is not in the vocabulary, it is split into UTF-8 bytes and represented by special byte tokens. This eliminates OOV entirely but can dramatically increase sequence lengths for scripts like Chinese, Korean, or Arabic, where characters may decompose into multiple bytes.

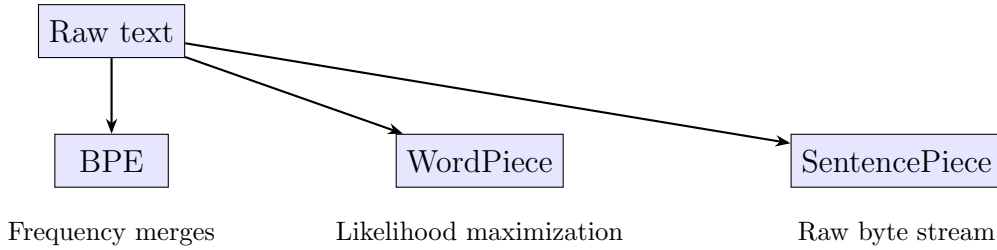


Figure 5: Three dominant tokenization algorithms. BPE uses frequency-based merges; WordPiece maximizes a likelihood criterion; SentencePiece operates directly on bytes without pre-tokenization.

4.2 KV-Cache Dynamics in Autoregressive Generation

Autoregressive generation produces tokens one by one. Without caching, at step t the Transformer recomputes keys and values for all previous $t - 1$ tokens, leading to a total cost proportional to $O(T^2)$.

The KV-cache stores the key and value projections of all processed tokens. At step t , only the new token’s query, key, and value are computed. The attention then uses the cached $\mathbf{K}_{1:t-1}$ and $\mathbf{V}_{1:t-1}$:

$$\mathbf{a}_t = \text{softmax}\left(\frac{\mathbf{q}_t[\mathbf{K}_{1:t-1}; \mathbf{k}_t]^\top}{\sqrt{d_k}}\right) [\mathbf{V}_{1:t-1}; \mathbf{v}_t]. \quad (9)$$

The computation per new token becomes $O(t)$ (since we multiply a single query against t keys), but the matrix multiplication is dominated by a small constant factor. Moreover, the compute is independent of the number of layers, and each step adds a constant-sized new $\mathbf{k}_t, \mathbf{v}_t$ to the cache. Thus the per-step compute is effectively $O(1)$ in terms of scaling with t , though the memory footprint grows linearly with the sequence length T , as we must store two matrices of size (T, d_{model}) per layer.

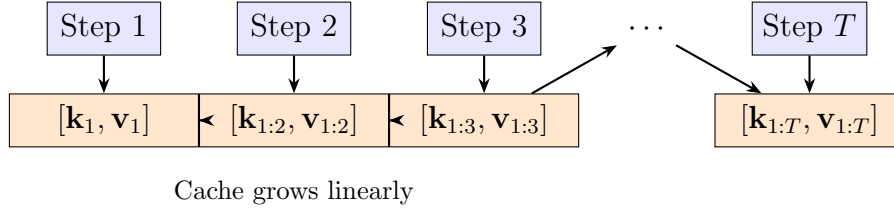


Figure 6: KV-cache mechanism. At each step, the new key and value are appended to the cache. The attention computation uses all cached keys and values, but only a single new query. Compute per step is $O(1)$, memory grows as $O(T)$.

4.3 Code Implementation: KV-Cache Usage in Attention

Listing 3 demonstrates how a KV-cache is used inside a single attention layer during autoregressive generation.

```

1 def attention_with_cache(Q_new, K_new, V_new, cache_K,
2   cache_V):
3     """
4     Q_new: (batch, 1, d_k) -- query for the new token only
5     K_new: (batch, 1, d_k) -- key for the new token
6     V_new: (batch, 1, d_v) -- value for the new token
7     cache_K: (batch, t-1, d_k) -- past keys
8     cache_V: (batch, t-1, d_v) -- past values
9     Returns: (output, updated_cache_K, updated_cache_V)
10    """
11    # Concatenate past with new
12    K_full = torch.cat([cache_K, K_new], dim=1)
13    V_full = torch.cat([cache_V, V_new], dim=1)
14    # Compute attention
15    d_k = Q_new.size(-1)
16    scores = torch.matmul(Q_new, K_full.transpose(-2, -1)) /
17              (d_k ** 0.5)
18    attn_weights = torch.softmax(scores, dim=-1)
19    output = torch.matmul(attn_weights, V_full)
20    return output, K_full, V_full

```

Listing 3: Simplified attention with KV-cache during inference.

5 Conclusion

This report has dissected the key engineering and mathematical components that underpin modern Transformers. The architectural routing choices—bidirectional

(BERT), causal (GPT), or encoder-decoder (T5)—define the fundamental capabilities and limitations of a model. Parameter-efficient techniques like LoRA and QLoRA capitalize on the low intrinsic rank of weight updates, drastically cutting fine-tuning costs while maintaining performance. Tokenization granularity, governed by BPE, WordPiece, or SentencePiece, critically impacts sequence length and model compactness. Finally, inference optimizations such as the KV-cache transform the autoregressive generation from a quadratic bottleneck into a linear-memory, constant-compute step. The interaction of these factors shapes the entire LLM lifecycle, from training to deployment.

References

- [1] A. Vaswani *et al.*, “Attention is all you need,” in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [2] J. Devlin *et al.*, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [3] A. Radford *et al.*, “Improving language understanding by generative pre-training,” OpenAI, 2018.
- [4] C. Raffel *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.
- [5] E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models,” in *Int. Conf. Learn. Represent.*, 2022.
- [6] T. Dettmers *et al.*, “QLoRA: Efficient finetuning of quantized LLMs,” in *Adv. Neural Inf. Process. Syst.*, 2023.
- [7] T. Kudo and J. Richardson, “SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” in *Proc. EMNLP (System Demonstrations)*, 2018, pp. 66–71.
- [8] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proc. ACL*, 2016, pp. 1715–1725.
- [9] Y. Wu *et al.*, “Google’s neural machine translation system: Bridging the gap between human and machine translation,” *arXiv preprint arXiv:1609.08144*, 2016.